HW4 - ECON460

Mazen Abouelela, Justin Chen, William An

2025-12-03

```r
set.seed(0)
#install.packages('pROC')
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
library(ggplot2)
```

1. (Classification) Consider the spam example from lecture 3 for a classification problem.

```r
spam <- read.csv("/Users/mazenabouelela/Downloads/spam.csv")
```

(a) Estimate the binary logit specification on the full data set and predict spam vs not-spam from this logit regression. Then plot an ROC curve for this classification. Note: Use the roc function from pROC package, which is a more comprehensive function than the roc function written by Matt Taddy. The syntax is: roc(response=truecategory,predictor=predictedprobability,plot=TRUE)

```r
# Fit binary logit on full dataset
logit_full <- glm(spam ~ ., data = spam, family = binomial(link = "logit"))
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
# Predicted probabilities + 0.5-threshold class predictions
pred_prob_in <- predict(logit_full, type = "response")
pred_class_in <- ifelse(pred_prob_in >= 0.5, 1, 0)

# classification results
table(True = spam$spam, Predicted = pred_class_in)
```

```
##     Predicted
## True    0    1
##    0 2669  119
##    1  167 1646
```

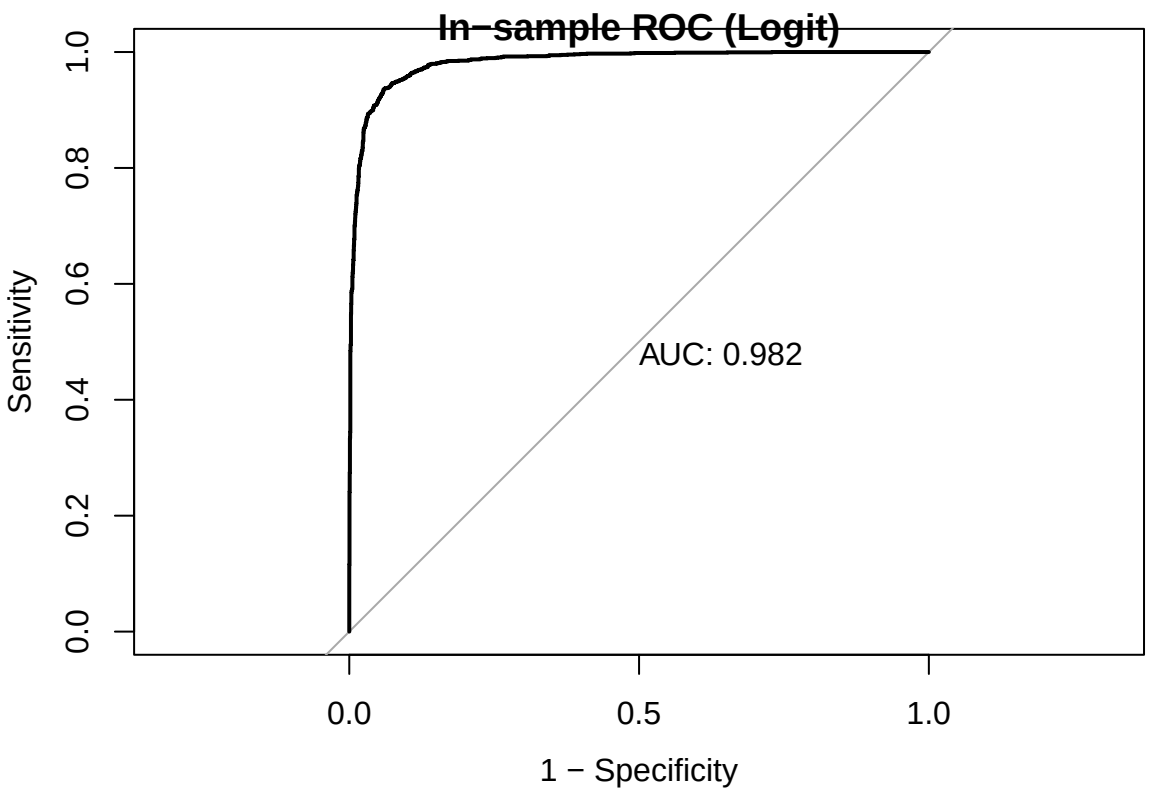
```r
mean(pred_class_in != spam$spam)  # in-sample classification error rate
```

```
## [1] 0.0621604
```

```r
# In-sample ROC curve
roc_in <- roc(response = spam$spam, predictor = pred_prob_in, plot = TRUE, legacy.axes = TRUE, print.au
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
title("In-sample ROC (Logit)")
```



(b) Plot an out-of-sample ROC curve by estimating the same logit specification on 4/5 of the data (sampled randomly), and evaluating classification error on the remaining 1/5 of the data.

```r
n <- nrow(spam)
train_idx <- sample(seq_len(n), size = floor(0.8 * n))
train <- spam[train_idx, ]
test  <- spam[-train_idx, ]

# Same logit specification, fit on 4/5
logit_train <- glm(spam ~ ., data = train, family = binomial(link = "logit"))
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
# Predict on held-out 1/5
pred_prob_oos <- predict(logit_train, newdata = test, type = "response")
pred_class_oos <- ifelse(pred_prob_oos >= 0.5, 1, 0)

# Out-of-sample classification error
table(True = test$spam, Predicted = pred_class_oos)
```

```
##     Predicted
## True   0   1
##    0 550  29
##    1  38 304
```

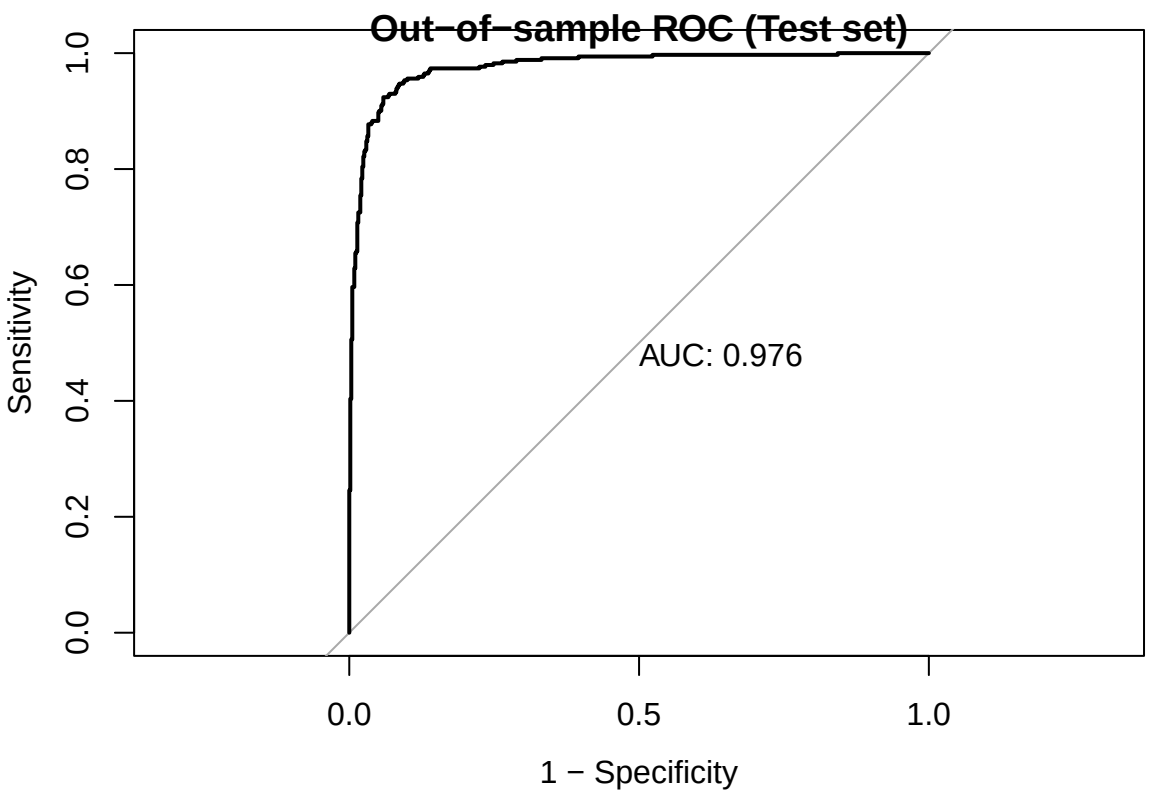
```r
mean(pred_class_oos != test$spam)
```

```
## [1] 0.07274701
```

```r
# Out-of-sample ROC curve (test set)
roc_oos <- roc(response = test$spam, predictor = pred_prob_oos, plot = TRUE, legacy.axes = TRUE, print.a
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
title("Out-of-sample ROC (Test set)")
```



Area Under the Curve (AUC) calculates the area under the ROC curve, representing the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative. Given the 45 degree line is the worst case (obtained by completely random classification), AUC ranges from 0.5 (worst case) to 1 (perfect case). You can calculate this measure by calling the auc from the roc object you got above (e.g., roccurve$auc).

(c) Calculate this for both in-sample ROC curve and out-of-sample ROC curve and compare between the two, what do you find? What do you think are pros and cons using AUC to compare models? (Hint: AUC is an 'aggregate' measure of a model, without specifying a threshold i.e., without considering a specific cost function.)

```r
auc_in  <- auc(roc_in)
auc_oos <- auc(roc_oos)

auc_in
```

```
## Area under the curve: 0.9825
```

```r
auc_oos
```

```
## Area under the curve: 0.9764
```

```r
cat("AUC (in-sample):     ", round(as.numeric(auc_in), 4), "\n")
```

```
## AUC (in-sample):      0.9825
```

```r
cat("AUC (out-of-sample): ", round(as.numeric(auc_oos), 4), "\n\n")
```

```
## AUC (out-of-sample):  0.9764
```

The in-sample AUC (~0.9825) is slightly higher than the out-of-sample AUC (~0.9764), which is what we expect: the model fits best on the data it was trained on but still performs almost as well on new data. The small gap between them is consistent with a model that's powerful but not severely overfit.

Using AUC to compare models has some advantages: it summarizes performance across all possible thresholds, it is insensitive to the exact choice of cutoff, and it captures how well a model ranks positive cases above negative ones. However, AUC also has drawbacks. It does not incorporate the actual costs of false positives and false negatives or the particular threshold you plan to use in practice, so two models with similar AUCs can behave very differently in the region of the ROC curve that corresponds to the preferred operating point. AUC is therefore a useful "aggregate" measure of discriminatory power, but it should be complemented with threshold-specific metrics when making real decisions.

4. (PCA) We will apply principal component analysis to congressional voting data. We will use data on Congressional Record for roll-call votes for the 445 representatives of the 111th Congress (2009-2010). These are votes where attendance is taken and every individual's vote is added to the Congressional Record. We will apply PCA to see if it uncovers a lower-dimensional structure to the d = 1647 dimensional vector of congressperson's votes.

```r
votes <- read.csv("rollcall-votes.csv")
members <- read.csv("rollcall-members.csv")
```

(a) Read the rollcall-votes.csv (votes data) and rollcall-members.csv (some variables about the congress members) in. In the votes data, -1 means 'nea', +1 means 'yea', and 0 means 'abstained'. Take a look at the data.

```r
dim(votes)
```

```
## [1]  445 1647
```

```r
dim(members)
```

```
## [1] 445   3
```

```r
head(votes[, 1:10])
```

```
##                     Vote.1 Vote.2 Vote.3 Vote.4 Vote.5 Vote.6 Vote.7 Vote.8
## BONNER (R AL-1)         -1      1     -1      0      0      1     -1     -1
## BRIGHT (D AL-2)          1     -1      1      1      1      1     -1     -1
## ROGERS (R AL-3)         -1      1     -1     -1     -1      1     -1     -1
## ADERHOLT (R AL-4)       -1      1     -1     -1      1      1     -1     -1
## GRIFFITH (D/R AL-5)      1     -1      1      1      1     -1     -1     -1
## BACHUS (R AL-6)         -1      1     -1      1      1      1     -1      0
##                     Vote.9 Vote.10
## BONNER (R AL-1)          1       1
## BRIGHT (D AL-2)          1       1
## ROGERS (R AL-3)          1       1
## ADERHOLT (R AL-4)        1       1
## GRIFFITH (D/R AL-5)      1       1
```

```
## BACHUS (R AL-6)          1       1
```

```r
head(members)
```

```
##                member state party
## 1     BONNER (R AL-1)    AL     R
## 2     BRIGHT (D AL-2)    AL     D
## 3     ROGERS (R AL-3)    AL     R
## 4   ADERHOLT (R AL-4)    AL     R
## 5 GRIFFITH (D/R AL-5)    AL    DR
## 6     BACHUS (R AL-6)    AL     R
```

```r
# Distribution of vote codes (-1, 0, +1)
table(as.vector(as.matrix(votes)))
```

```
##
##     -1      0      1
## 178599  53191 501125
```

```r
# Party counts
table(members$party)
```

```
##
##   D  DR   R
## 262   1 182
```

```r
# check that rows align
stopifnot(nrow(votes) == nrow(members))
```

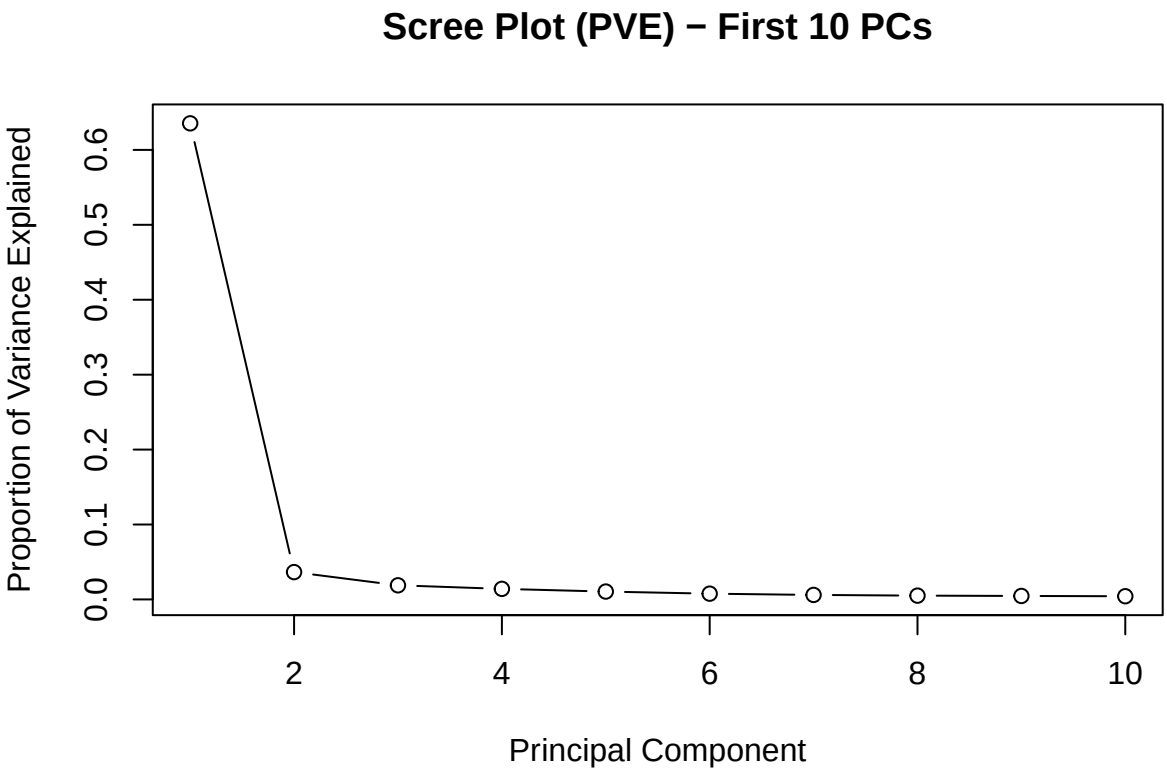
(b) Run PCA with K = 10. Draw the scree plot and investigate the proportion of variance explained. How much variance do the first and second principal components explain?

```r
vote_mat <- as.matrix(votes)

pca <- prcomp(vote_mat, center = TRUE, scale. = FALSE)

# Proportion of variance explained (PVE)
pve <- (pca$sdev^2) / sum(pca$sdev^2)

# Scree plot for first K = 10 components
plot(pve[1:10], type = "b",
     xlab = "Principal Component",
     ylab = "Proportion of Variance Explained",
     main = "Scree Plot (PVE) - First 10 PCs")
```




```r
cat("Variance explained by PC1:", round(100 * pve[1], 2), "%\n")
```

```
## Variance explained by PC1: 63.54 %
```

```r
cat("Variance explained by PC2:", round(100 * pve[2], 2), "%\n")
```

```
## Variance explained by PC2: 3.65 %
```

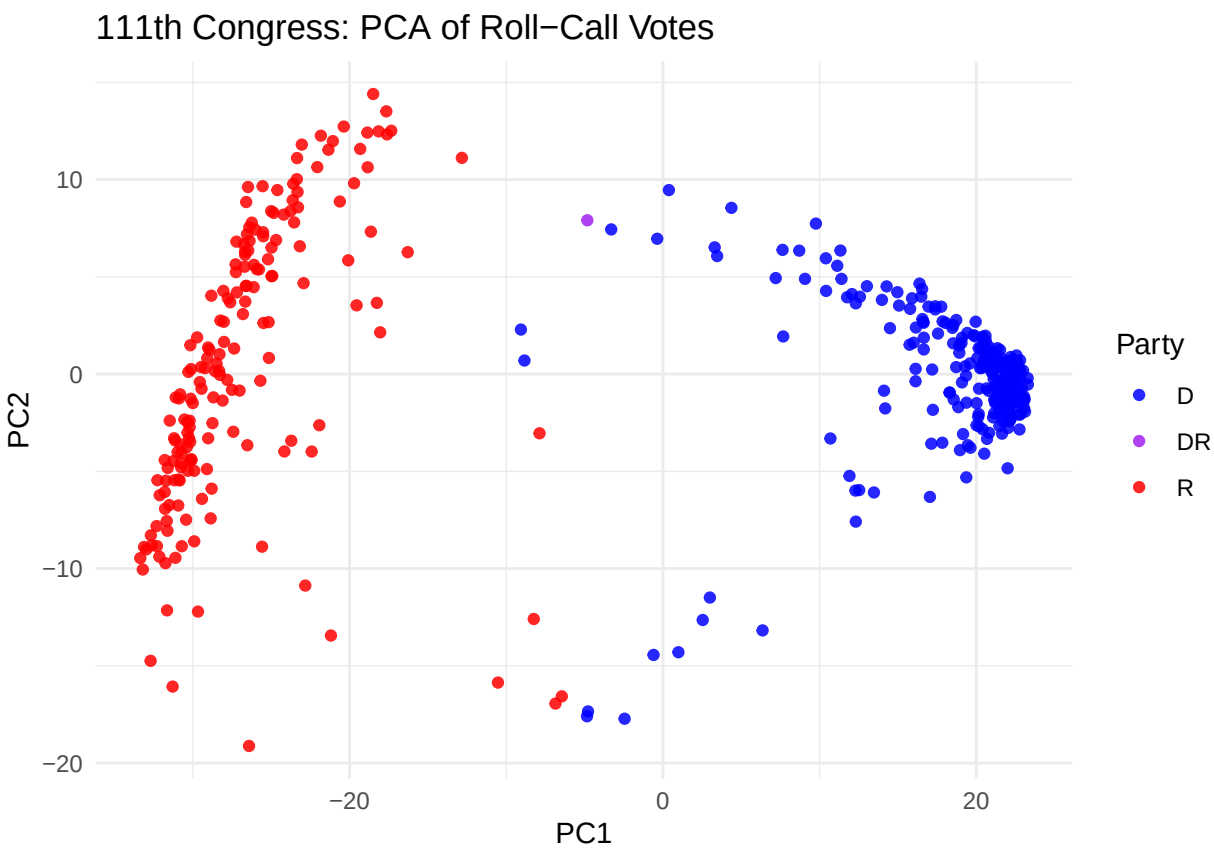
(c) Get the estimated latent factors by predict and plot the first factor on the x-axis and the second factor on the y-axis. Color by the party of congressperson. How would you interpret the first principal component?

```r
# Latent factors / scores (first 2 PCs)
scores <- predict(pca)[, 1:2]

pc_df <- data.frame(
  PC1 = scores[, 1],
  PC2 = scores[, 2],
  party = factor(members$party),
  member = members$member,
  state = members$state
)

ggplot(pc_df, aes(x = PC1, y = PC2, color = party)) +
  geom_point(alpha = 0.85) +
  theme_minimal() +
  labs(
    title = "111th Congress: PCA of Roll-Call Votes",
    x = "PC1",
    y = "PC2",
    color = "Party"
```

```r
  ) +
  scale_color_manual(
    values = c(
      "D"  = "blue",
      "R"  = "red",
      "DR" = "purple"
    )
  )
```




```r
# numeric support for interpreting PC1 (sign can flip, but separation matters)
aggregate(PC1 ~ party, data = pc_df, mean)
```

```
##   party        PC1
## 1     D  18.493033
## 2    DR  -4.824641
## 3     R -26.595330
```

Plotting PC1 against PC2 and coloring points by party reveals a strong separation along the horizontal (PC1) axis: Republicans cluster on one side (negative PC1 values), Democrats cluster on the other side (positive PC1 values), and the lone DR member sits in between. The party-wise means of PC1 confirm this pattern: Democrats have a large positive mean PC1, Republicans a large negative mean, with DR in the middle. PC2 mostly reflects within-party variation and is comparatively less important. This pattern supports interpreting PC1 as the main partisan or ideological dimension of voting in the 111th Congress—a left–right axis separating Democrats from Republicans based on their overall voting records.

(d) Take a look at 5 people with the largest first principal component, and 5 people with the smallest first principal component. Does it support your story in (c)?

```r
pc_sorted <- pc_df[order(pc_df$PC1), ]

# 5 smallest PC1
head(pc_sorted[, c("member", "state", "party", "PC1", "PC2")], 5)
```

```
##                                member state party       PC1        PC2
## LAMBORN (R CO-5)     LAMBORN (R CO-5)    CO     R -33.35849  -9.462467
## HENSARLIN (R TX-5) HENSARLIN (R TX-5)    TX     R -33.20087 -10.048553
## FOXX (R NC-5)           FOXX (R NC-5)    NC     R -33.11785  -8.888623
## FRANKS (R AZ-2)       FRANKS (R AZ-2)    AZ     R -32.97302  -9.010658
## BROUN (R GA-10)       BROUN (R GA-10)    GA     R -32.70210 -14.743409
```

```r
# 5 largest PC1
tail(pc_sorted[, c("member", "state", "party", "PC1", "PC2")], 5)
```

```
##                              member state party      PC1        PC2
## HIRONO (D HI-2)     HIRONO (D HI-2)    HI     D 23.05447 -1.1579998
## SCHAKOWS (D IL-9) SCHAKOWS (D IL-9)    IL     D 23.09021 -1.2986579
## EDWARDS (D MD-4)   EDWARDS (D MD-4)    MD     D 23.11430 -1.9300268
## PRICE (D NC-4)       PRICE (D NC-4)    NC     D 23.28314 -0.2067589
## MATSUI (D CA-5)     MATSUI (D CA-5)    CA     D 23.29481 -0.5354906
```

Looking at the five representatives with the smallest PC1 scores, they are all Republicans, while the five representatives with the largest PC1 scores are all Democrats. These members at the extremes can be thought of as the most consistently conservative vs. the most consistently liberal voters according to the data. This pattern of party affiliation at the tails of PC1 is consistent with and reinforces the interpretation from part (c): PC1 captures the main ideological/partisan divide in congressional voting behavior.